target = 5.

7 1 2 2 8 4 1 5 8

cnt = ~~x~~ 3

TC = O(n)
AS = O(1)

---

tac, act, cat ✓

acct, cat x

abc, def x

I/P = [act, god, cat, dog, tac].

O/P = [[act, cat, tac],
[god, dog]].

Hash Map.

something common b/w anagrams → [list of strings].

act → [act, cat, tac]
dgo → [god, dog].

→ TC = O(n mlogm)
AS = O(nm).

[①, 2, 3]
→ ["geeksforgeeks", "hello", "hi"].

Hash Map.

freq Array → [list of anagrams].

string version of freq Array → [list of anagrams].

act cat.

act | a 1 | b 0 | c 1 | ... | t 1 | ...
cat | 1 | 0 | 1 | ... | 1 | ...

"~~1010000 ... 1000~~"

~~a c → 0+2~~
~~b b → ++1~~

"1#0#1#0 ...... #1#0 ..."

a b
11 # 1 #
1 # 11 #

TC : O(nm).
AS : O(nm).

key → []

[Key → Hash func → int.] class Transport {

0 1 2 3 4
[act, god, cat, dog, tac].
fs1 fs2

fs1 → [act, cat, tac] AS = O(nm)
fs2 → [god, dog].

fs1 → [0, 2, 4]
fs2 → [1, 3]. AS = O(n).

Recursion.

function calling itself over & over again until it hits some stopping condition. (Base conditions)

func(int) {
- - -
func(int-1)
- -
} MAIN

func(5) {
func(4)
}

func(4) {
func(3)
}

func(3) {
func(2)
}

func(2) {
func(1)
}

func(1) {
func(0)
}

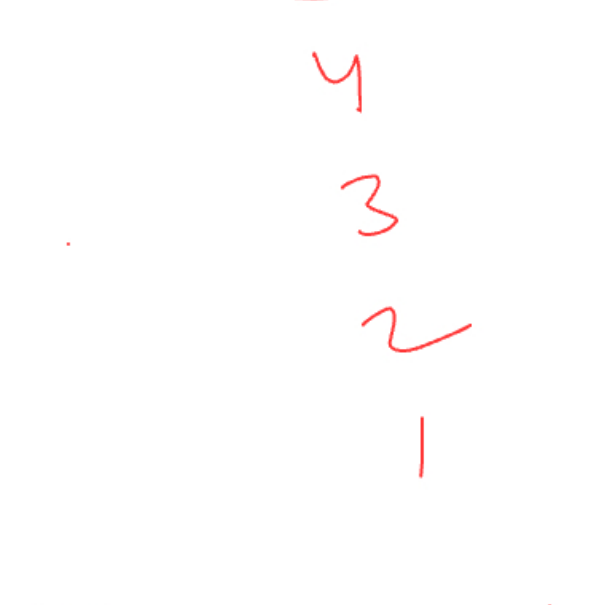
func(0) {
STOP!

}

Backtracking.

N to 1.

N - 1
N-1 - 1
~~N-2 - 1~~ X



```
void printNto1(int n) {
    if (n <= 0) {
        return;
    }

    cout << n << endl;
    printNto1(n - 1);
}
```

n = 5
n = 4
n = 3
n = 2
n = 1
n = 0

STDOUT.
5
4
3
2
1

factorial (n). {
Base Cond...

return n x factorial(n-1);

}

Not tail Recursive.

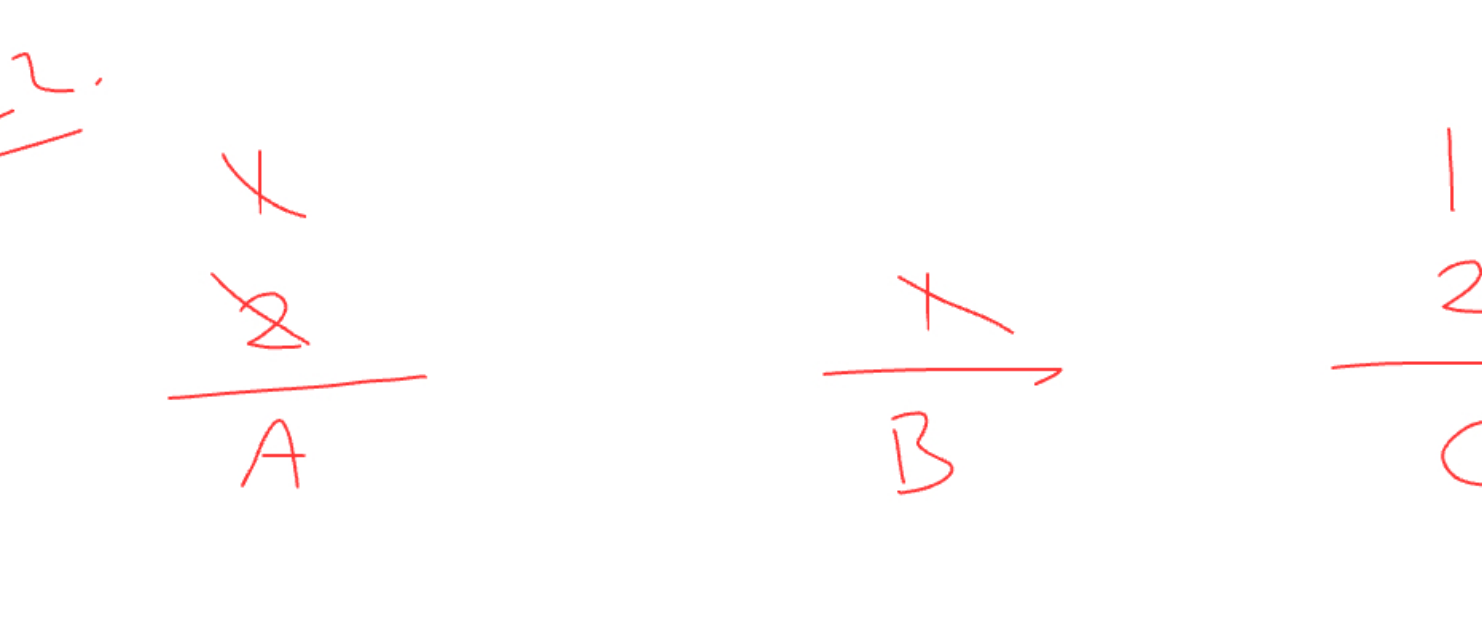
TODO: Make this tail recursive.

(~~i,~~ n) f(&res, n) {

res *= n.
f(res, n-1)

}

Tower of Hanoi.



n=2.

~~1~~
~~2~~
A

~~1~~
B

1
2
C

|||

n=3.

~~1~~
~~1~~
~~2~~
~~3~~
A

~~1~~
~~2~~
B

1
2
3
~~1~~
C

𝍸 ||